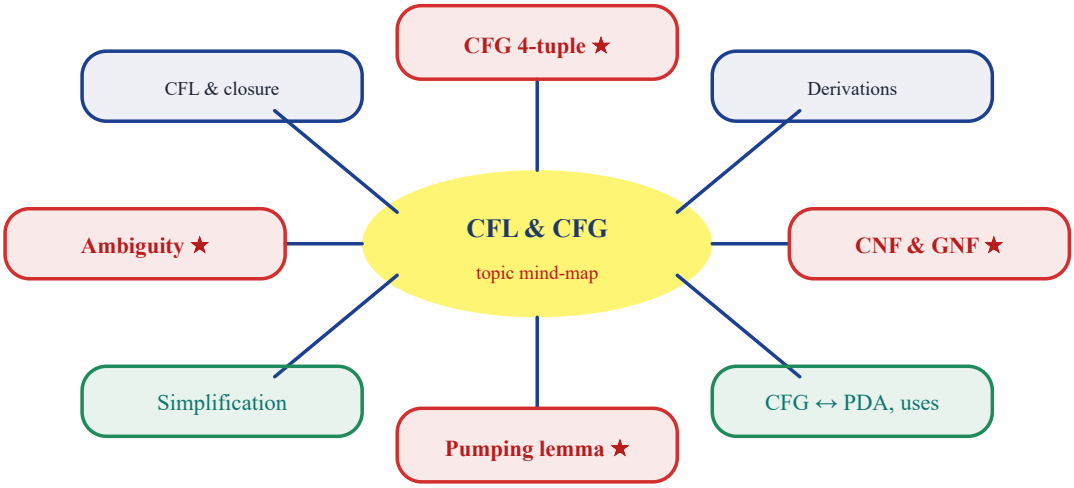
Context-Free Languages & CFG

CFL · CFG 4-tuple · derivations · ambiguity · CNF/GNF · pumping lemma · PDA: in 10 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

"Language theory; Finite automata: …pushdown automata; **Context free languages**; **Context free grammars**; Turing Machines: basic machines, configuration, computing with Turing machines." This notebook carries the **context-free languages and grammars**; the PDA was in the automata lesson, Turing machines come next.

High-yield list

① CFG = 4-tuple (V, Σ, P, S)

② one variable on every left side = CF

③ S → aSb | ε gives $a^n b^n$ with **n ≥ 0**

④ CNF = A → BC or A → a; GNF = A → aα

⑤ CFL closed under ∪, concat, * — **not** ∩ · but CFL ∩ regular = CFL

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Context-free languages …… p. 2
✓ The CFG 4-tuple …… p. 3
✓ Derivations, parse trees, ambiguity …… p. 4
✓ Worked grammar: balanced parentheses …… p. 5
✓ Simplifying a CFG: CNF & GNF …… p. 6
✓ Pumping lemma, CYK, step counts …… p. 7
✓ CFG ↔ PDA and applications …… p. 8
✓ Night-before sheet …… p. 9
✓ Practice MCQs + written …… p. 10

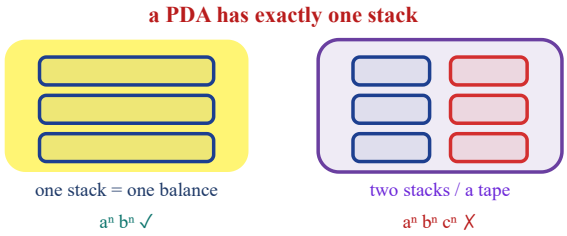
1 Context-Free Languages

Definition: A language is **context-free** when a **context-free grammar** can **generate** it and a **pushdown automaton** can recognise it — Chomsky **Type 2**. A stack holds one nested structure at a time, and no more.

Context-free language	What it says	Grammar
$\{a^n b^n \mid n \geq 0\}$	as many a's as b's	$S \to aSb \mid \varepsilon$
$\{w\, w^R \mid w \in \{a,b\}^*\}$	every palindrome over {a,b}	$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$
$\{a^n b^m c^m \mid n,m \geq 0\}$	one free block, then two matched	$S \to AB \cdot A \to aA \mid \varepsilon \cdot B \to bBc \mid \varepsilon$
$\{ (^n)^n \mid n \geq 0 \}$	balanced parentheses	$S \to (S) \mid SS \mid \varepsilon$
$\{a^n b^n c^n\}$	**not** context-free	needs two stacks

Closure properties — the whole table

Operation on languages	CFL closed?
union	**yes**
concatenation	**yes**
Kleene star	**yes**
complement	**no**
intersection	**no**
intersection with a **regular** language	**yes**



one stack counts one balance, so $a^n b^n c^n$ is out of reach

Why each answer: union: a new start symbol chooses either grammar · concatenation: $S_1 \to S_2$, then L_2's rules · star: $S \to SS \mid \varepsilon$, then L's rules · complement: flipping the final states of a PDA yields no CFG · intersection with a regular language: run the PDA, then the finite automaton on top.

Exam trap: "Not closed under intersection" needs its proof: $L_1 = \{a^n b^n c^m\}$ and $L_2 = \{a^m b^n c^n\}$ are context-free, but $L_1 \cap L_2 = \{a^n b^n c^n\}$ is **not**. So "not closed under ∩" means **not in general** — yet CFL ∩ **regular** is always a CFL.

Memory trick: Regular = ∪ ∩ ¬ * · CFL = ∪ · *. The only closure regular languages have that CFLs lack is **intersection**.

```
aⁿ bᵐ cᵐ :  S -> A B ,   A -> a A | e ,   B -> b B c | e
"aabbcc" :  S => AB => aAB => aaAB => aaB => aabBc => aabbBcc => aabbcc
```

2 Context-Free Grammar: the 4-Tuple

Definition: A **context-free grammar** is G = (**V**, **Σ**, **P**, **S**) — variables, terminals, productions, start symbol. It generates L(G) by repeatedly replacing a variable with the right-hand side of one of its productions. It is context-free exactly when **exactly one variable on the left**: nothing ever sits beside it.

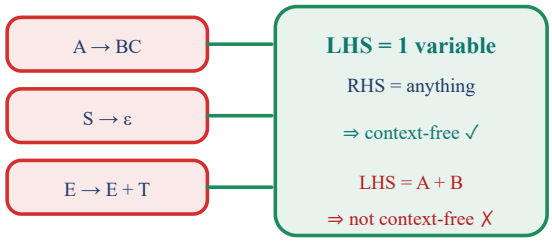
Part	Is	In the example G
V	finite set of **variables**; S ∈ V	V = {S, A, B}
Σ	finite set of **terminals**; V ∩ Σ = ∅	Σ = {a, b}
P	set of **productions** A → α, α ∈ (V ∪ Σ)*	P = { S → aSb, S → ε }
S	the **start symbol**, S ∈ V	S = S

The grammar for {$a^n b^n$ | n ≥ 0}

```
G = ( {S}, {a, b}, P, S )
P = { S -> a S b ,  S -> e }

leftmost derivation of "aaabbb":
S => aSb => aaSbb => aaaSbbb => aaabbb
```

Each use of **S → aSb wraps one more a on the left and one b on the right**; **S → ε** stops it. After n uses you have **$a^n b^n$**, and n = 0 gives **ε** — so the language is **{$a^n b^n$ | n ≥ 0}**, **not** n ≥ 1.



the whole definition in one picture: one variable on the left

Notation	Means
a, b, 0, 1	**terminal** — a real character
S, A, T	**variable** — a placeholder to replace
A → α	a **production**
A ⇒ β	one **derivation** step
A ⇒* w	zero or more steps, w all terminals
ε	the empty string, |ε| = 0

Memory trick: Four letters in order: **V**ariables, the **Σ**ymbols they sit on, the **P**roductions, the **S**tart — "V–Σ–P–S".

Exam trap: The condition sits on the **left**, never on the right. `A → aSb` is context-free; `AB → aSb` is not, whatever its right side looks like.

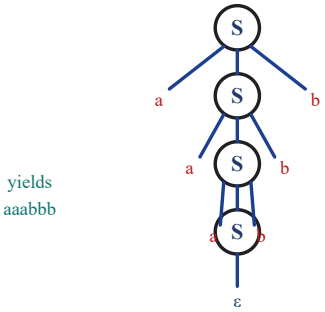
IT link: Every programming language ships a CFG in **BNF** as the **definition** of its syntax; a parser generator turns that grammar into the tables that validate a program.

> V ∩ Σ = ∅ — a variable is never also a terminal, so the four parts never collide

3 Derivations, Parse Trees and Ambiguity

Derivation: Start from **S**, replacing one variable at a time until only terminals remain. A **leftmost** derivation always replaces the leftmost remaining variable, a **rightmost** one the rightmost. Both reach the same string — only the **order of work** differs, which is why parsers are built leftmost.

```
LEFTMOST          RIGHTMOST
S                   S
S => aSb          S => aSb
S => aaSbb        S => aSbb
S => aaaSbbb      S => aaSbb
S => aaabbb       S => aaabbb
```



yields aaabbb

parse tree of S ⇒ aSb ⇒ aaSbb ⇒ aaaSbbb ⇒ aaabbb

Parse tree = the derivation, drawn: Every variable is a node; every applied production is a node whose children are that rule's right-hand symbols, left to right. **leaves read left to right** give the derived string, so **leaves = the string** — and for n terminals the tree has exactly **n − 1** internal nodes.

Ambiguity: two trees for one string

Definition: A grammar is **ambiguous** if **some** string has **two or more distinct leftmost derivations** — equivalently two parse trees. One bad string is enough.

Worked example ✍️ the classic ambiguous grammar:

```
E -> E + E | E * E | id

tree 1: (id+id)*id          tree 2: id+(id*id)
        *                           +
       / \                         / \
      +   id                     id   *
     / \                            / \
   id   id                        id  id
```

Same four terminals, two groupings — so **id+id*id** is ambiguous. Fixing it needs one extra level per operator: **E → E + T | T, T → T * F | F, F → id**.

Memory trick: "Ambiguous = **a**lternative tree". If one string can be **built two ways**, the parser cannot know what was meant.

Exam trap: Ambiguity has nothing to do with ε-productions, with the number of variables, or with an infinite language. And it is not nondeterminism: a PDA is nondeterministic on **every** CFL.

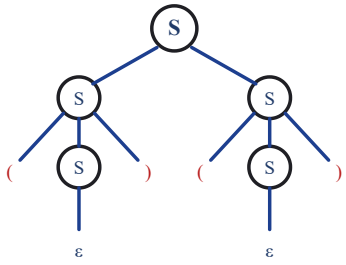
4 Worked Grammar: Balanced Parentheses

Definition: The language of **balanced** parentheses over $\Sigma = \{ (,) \}$ is $\{ (^n)^n \mid n \geq 0 \}$: every (is closed by a later), and the nesting is proper. It is context-free, and it is what the pushdown automaton from the automata lesson accepts by pushing on (and popping on).

The grammar

```
V = {S},  Sigma = { ( , ) },  S = S
P = { S -> SS ,      two groups side by side
      S -> (S) ,      one group inside another
      S -> e }        the empty group

leftmost derivation of "(())()":
S => SS => (S)S => ((S))S => (())S => (())(S) => (())()
```



parse tree of (())(): two sibling groups, each empty inside

Read it as a proof of acceptance: every (is closed by exactly one), and no) comes first. The converse holds too — **S → SS** builds the first group and appends the rest, **S → (S)** takes care of one nesting level.

Production	The idea it encodes	Strings it can finish
S → ε	the base case: the empty group	ε, and only ε
S → (S)	wrap **one** group in one pair	( ), (( )), ((())) …
S → SS	concatenate two balanced groups	()(), (())(), (()()) …

Exam trap: Write **S → SS**, never **S → SS | SS**. The order of alternatives is irrelevant — a CFG's productions are a **set**, not a program: any rule may fire at any step. This is also why $\{a^n b^n c^n\}$ cannot be built this way: one stack cannot close one bracket while a second count is still open.

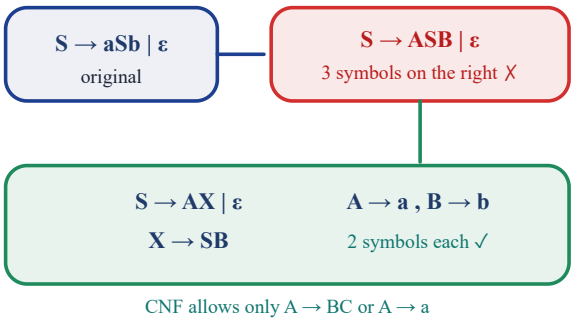
Memory trick: **SS = side by side, (S) = inside, ε = nothing.** The same three-rule shape builds balanced {}, balanced **[ ]**, and — with a third pair — balanced **{ } [ ]**.

IT link: An editor's bracket matcher is a PDA: push on an opening symbol, pop on the matching one, refuse to pop an empty stack. In a compiler the same grammar becomes the parser's production rules.

```
input:  (  (  )  )  (  )
stack:  Z0 -> Z0 -> (Z0 -> ((Z0 -> (Z0 -> Z0 -> (Z0 -> Z0   (accept)
```

5 Simplifying a CFG: CNF and GNF

1. **Remove useless symbols:** the **non-generating** variables, then the **unreachable** ones.
2. **Remove ε-productions** A → ε: find every **nullable** variable and drop its ε-rule.
3. **Remove unit productions** A → B by copying B's productions in their place (repeat — chains occur).
4. **Convert to Chomsky Normal Form** — what CYK and the parser generators demand.
5. **Convert to Greibach Normal Form**, where every rule starts with a terminal.



where the naive CNF conversion goes wrong, and the fix

Normal form	Every production is	Buys
CNF	**A → BC** or **A → a**; S → ε only when ε ∈ L	CYK, yacc/bison
GNF	**A → aα**	one-symbol prediction
right-linear	A → aB	a **regular** Type 3 grammar

Worked conversion ✍️ S → aSb | ε into CNF:

```
original   S -> a S b | e
step 1     wrap the left terminal, then the right:  S -> A S B
step 2     THREE symbols on the right, so split the tail:
           S -> A X   ,   X -> S B   ,   A -> a   ,   B -> b
result     S -> A X | e ,  X -> S B ,  A -> a ,  B -> b
```

Why S → ε survives: CNF allows it only when ε ∈ L **and** S appears on no right-hand side. Here S **does**, inside **X → SB**, so the strictly correct form adds a fresh start symbol **S₀ → S** and keeps the other four rules.

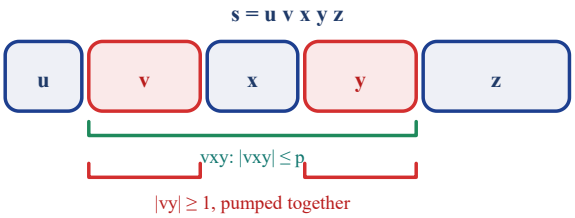
Exam trap: *S → ASB* is **not** CNF — exactly **two** variables or **one** terminal, never three symbols. And *A → aα* is **Greibach**, not CNF.

Memory trick: **CNF = two or one, GNF = starts with a terminal.** Count the right-hand symbols: 2 or 1 ⇒ CNF; "a then variables" ⇒ GNF; "aB" ⇒ regular.

6 Pumping Lemma for CFLs, CYK and Step Counts

Pumping lemma for context-free languages: If L is context-free then some length **p** exists such that every s ∈ L with |s| ≥ p splits as **s = uvxyz** with **|vxy| ≤ p**, **|vy| ≥ 1** and **$uv^ixy^iz \in L$ for every i ≥ 0**. Read it as: inside a short window lies a balanced piece that may be repeated as often as you like. Like the regular version it is a **disproof** tool only.

Condition	Says
$\|vxy\| \le p$	the pumpable window is **short**
$\|vy\| \ge 1$	v and y together are **non-empty**
$uv^ixy^iz \in L, \forall i \ge 0$	v and y repeat **together**



the five-piece split of the pumping lemma for CFLs

Exam trap: The regular lemma pumps xy^nz; the CFL lemma pumps **uv^ixy^iz** — **two** exponents, and they are **equal**. Writing $u(vxy)^nz$ is wrong. And satisfying the lemma never proves a language **is** context-free.

Worked disproof ✍ $\{a^n b^n c^n \mid n \ge 0\}$ is not context-free:
Take pumping length p and s = $a^pb^pc^p$. Every decomposition with |vxy| ≤ p and |vy| ≥ 1 puts y wholly inside the **a-block** ($y = a^k$, $k \ge 1$) or wholly inside the **b-block** ($y = b^k$). Pumping y alone breaks the match, and pumping v and y **together** cannot restore it, because they sit in different blocks. The condition fails ⇒ **L is not context-free**. In one line: it needs three simultaneous counts, and one stack holds one.

CYK and the step count: **CYK** decides membership for a **fixed CNF** grammar in **$O(n^3)$** time ($O(n^3 \cdot |G|)$ in general): a bottom-up table over $O(n^2)$ substrings, trying O(n) split points each. In CNF a string of length **n ≥ 1** takes exactly **2n − 1** steps — **n − 1** of A → BC to turn one variable into n variables, then **n** of A → a.

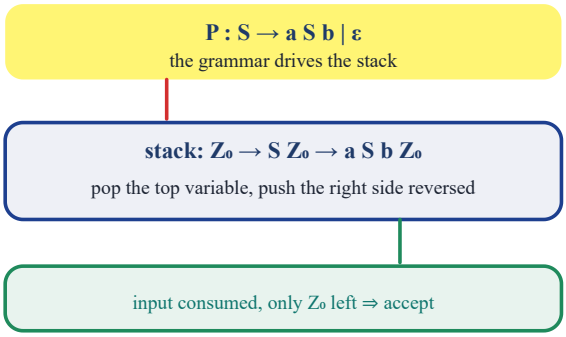
Memory trick: **"n − 1 splits, then n terminals = 2n − 1 altogether"** — and **CYK is cubic**: three nested loops over substrings, split points and rules.

> Unlike the regular lemma, the CFL lemma has no way to prove context-freeness — only to deny it

7 CFG ↔ PDA and Where CFGs Are Used

Equivalence theorem (high-yield): **context-free ⇔ some PDA**. "If" is trivial — read the grammar as a leftmost derivation. "Only if" is the surprise: the PDA's **states become variables**, so the machine writes its own grammar.

1. **Push the start symbol** S on the stack above the bottom marker Z_0.
2. If the **stack top is a variable** A with A → α in P, pop A and push α in **reverse** order.
3. If the **stack top is a terminal** a, match it with the next input symbol: equal → pop and advance; unequal → reject.
4. Accept when the input is exhausted **and** only Z_0 is left on the stack.
5. Reverse direction: a variable **[pXq]** for every state pair p, q and stack symbol X, **[pXq] → a** per transition, **S_0 → [q_0 Z_0 q]** per accepting state q.



CFG → PDA: the grammar literally drives the stack

Application	How the CFG is used
Programming languages	C, Java, Pascal syntax is defined in BNF/EBNF — the grammar **is** the definition
Compilers	**parser generators** (yacc, bison, ANTLR) read the grammar and emit the tables
Markup languages	XML / HTML nesting is a CFG; the validator is a PDA over the tag stream
Natural language	sentence structure as a grammar; parsing opens every NLP pipeline
Query languages	SQL's `SELECT` … `WHERE` … is a recursive-descent grammar

Memory trick: Push the **start symbol**, replace the **top of stack**, accept on a **bare stack**. The reverse conversion names its variables **[pXq]** — "from p, on X, to q".

Exam trap: In CFG → PDA the right-hand side is pushed **reversed**: the stack is last-in-first-out. Push α in written order and the PDA builds the string **backwards**.

IT link: Bracket matching, *if*/*for* nesting checks and JSON validation are the same PDA of sheet 5.

Night-Before-the-Exam Sheet

Fact sheet:

CFL = made by a CFG, read by a PDA

CFG = **(V, Σ, P, S)**

$V \cap \Sigma = \emptyset$; $S \in V$; $P \subseteq V \times (V \cup \Sigma)^*$

$S \to aSb \mid \varepsilon \Rightarrow \mathbf{\{a^n\, b^n \mid n \geq 0\}}$

$S \to SS \mid (S) \mid \varepsilon \Rightarrow$ balanced brackets

Parse-tree leaves = the string

$E \to E{+}E \mid E{*}E \mid id$ is ambiguous

Not closed: ∩ and complement

$\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$ = not CF

GNF: $A \to a\alpha$ · $A \to aB$ = regular

$uv^ixy^iz \in L\ \forall\ i \geq 0$ — equal exponents

CNF derivation of length $n \geq 1$: **$2n - 1$**

CFL is Chomsky **Type 2**

CF ⇔ one variable per left side

$A \Rightarrow$ one step · $A \Rightarrow^*$ w · ε = empty

$S \to aSa \mid bSb \mid a \mid b \mid \varepsilon \Rightarrow$ palindromes

Leftmost = leftmost variable first

Ambiguous ⇔ ≥ 2 leftmost derivations

Closed: ∪, concatenation, Kleene star

but CFL ∩ **regular is** a CFL

CNF: $A \to BC$ or $A \to a$ (+ $S \to \varepsilon$)

CFL pumping: s = **uvxyz**, $|vxy| \leq p$, $|vy| \geq 1$

CYK on a fixed CNF grammar: **$O(n^3)$**

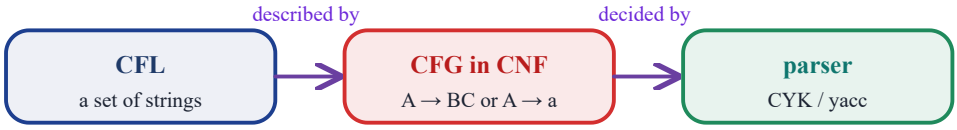
CFG ⇔ PDA; PDA → CFG uses **[pXq]**

Top 12 one-liners for MCQs

1. CFG = 4-tuple, PDA = 7-tuple
2. One variable on the left = context-free
3. $S \to aSb \mid \varepsilon$ gives **$n \geq 0$**, ε included
4. Type-3 grammars generate **regular** languages
5. CFL closed under ∪ · * — **not** ∩
6. CNF allows **only** $A \to BC$ or $A \to a$
7. $A \to a\alpha$ is **Greibach**, $A \to aB$ is right-linear
8. Ambiguous = one string, two leftmost derivations
9. The CFL lemma splits into **five** pieces uvxyz
10. CYK is **cubic** in the input length
11. The CNF step count is **$2n - 1$**
12. CFL ∩ regular is still a CFL

Top traps: a three-symbol right-hand side is **not** CNF · "not closed under intersection" needs the regular-language qualification · the CFL pumping lemma pumps v and y with the **same** exponent · satisfying a pumping lemma never proves context-freeness · a PDA pushes a production's right side in **reverse** order · an $n \geq 0$ language contains ε.

All tricks together: "V–Σ–P–S, four parts" · "one variable on the left is the whole definition" · "SS = side by side, (S) = inside, ε = nothing" · "leaves read left to right = the string" · "CNF = two or one, GNF = starts with a terminal" · "five pieces, two exponents" · "n − 1 splits plus n terminals = 2n − 1".



every step down = one more algorithm you can be asked about

same language, three descriptions — all interchangeable

✓ Practice MCQs (exam pattern, 10 marks)

1. A CFG is a tuple of how many parts?
(a) 3 (b) 4
(c) 5 (d) 7

2. In CNF every production has the form:
(a) $A \to a\alpha$ (b) $A \to aB$
(c) $A \to BC$ or $A \to a$ (d) any α, $|\alpha| \le 2$

3. Type-3 grammars generate:
(a) context-free (b) context-sens.
(c) regular (d) r. enum.

4. CFLs are **NOT** closed under:
(a) union (b) concat.
(c) Kleene star (d) intersect.

5. $S \to aSb \mid \varepsilon$ generates:
(a) $\{a^n b^n \mid n \ge 1\}$ (b) $\{a^n b^n \mid n \ge 0\}$
(c) $(ab)^*$ (d) a^*b^*

6. A grammar is ambiguous when:
(a) 2+ trees, 1 string (b) has ε-rule
(c) infinite lang. (d) >1 variable

7. CYK decides membership in:
(a) O(n) (b) O(n log n)
(c) $O(n^2)$ (d) $O(n^3)$

8. In CNF, deriving length $n \ge 1$ takes:
(a) n steps (b) $2n - 1$ steps
(c) 2n steps (d) n^2 steps

9. $\{a^n b^n c^n \mid n \ge 1\}$ is:
(a) regular (b) context-free
(c) neither, not CF (d) both

10. CFL ∩ a **regular** language is:
(a) always a CFL (b) no guarantee
(c) never a CFL (d) regular lang.

Answer key → 1-b · 2-c · 3-c · 4-d · 5-b · 6-a · 7-d · 8-b · 9-c · 10-a

Written question: Write a CFG for all palindromes over {a, b}; derive *abba* and *aba* leftmost; say why it is context-free but not regular. 10 marks

Model answer ✍️:

1. **Grammar** 3 — G = ({S}, {a,b}, P, S), P = { **S → aSa | bSb | a | b | ε** }: wrap both ends, finish the middle.
2. **Derivations** 3 — abba: S ⇒ aSa ⇒ abSba ⇒ **abba** · aba: S ⇒ aSa ⇒ **aba**.
3. **Why context-free** 2 — one variable per left side; an NPDA pushes the first half, **guesses** the middle and pops the second.
4. **Why not regular** 2 — with pumping length p, $w = a^p b a^p$ splits with $y = a^k$ in the **first** block, so xy^2z is not a palindrome.

Template to memorise: palindromes **S → aSa | bSb | a | b | ε** · brackets **S → SS | (S) | ε** · "wrap the ends, finish the middle".

Before the exam ✓

- ✓ Can write the 4-tuple (V, Σ, P, S) from memory
- ✓ Can give a grammar for $a^n b^n$, palindromes, balanced brackets
- ✓ Can convert a small grammar into CNF correctly